

G.A.I.G.S.
GLOBAL AI GOVERNANCE SYSTEM · ONEPIECEJOURNEY

FOR THE ENGINEERING TEAM · BUILD BLUEPRINT v1

Technical Master Plan

From working MVP to a real, production civic platform

Muhammad Qureshi — Founder
Naib Imam, Jamia Masjid-e-Nabawi Qureshi Hashmi · G-11/4 Islamabad, Pakistan
AI researcher · author · builder

Live: onepiecejourney-crew.netlify.app · WhatsApp +92 346 8053268 ·
linkedin.com/in/muhammad-qureshi-9939b1383

What this is & what's built

GAIGS is a people-owned civic operating system. Today it is a high-fidelity, fully client-side interactive product (HTML/CSS/JS) deployed on Netlify, with one serverless function proxying the AI (key server-side), an installable PWA, and a signed Android APK (TWA wrapping the live app — so every web update ships to the app instantly, no re-install). All state currently lives on-device (localStorage) — this proves the UX and the local-first idea, but it is NOT yet a multi-user networked system.

Layer	Current state
Frontend	Static HTML/CSS/vanilla JS; PWA; FB-light + ops-dark themes
AI	Netlify function -> Gemini (key server-side); voice + context-aware JARVIS
App	Signed APK (TWA of /gaigs/) + assetlinks.json; PWA install on web
Data	On-device localStorage (demo) — no shared backend/chain yet
Hosting	Netlify (site+function); optional Render/Oracle for 24/7 WhatsApp JARVIS

Target architecture (the real build)

2.1 Frontend (modernize)

- React Native + Expo + TypeScript for ONE Android/iOS app; React + Tailwind for web/desktop ops center.
- TanStack Query (server state), Zustand (client), Framer Motion; MapLibre; Recharts.
- PWA + offline-first; i18n with RTL Urdu/Arabic; SecureStore/Keychain/Keystore for keys.

2.2 Backend & data

- API: FastAPI (or NestJS). Auth (JWT + passkeys), real OTP (Twilio/local), RBAC + row-level security.
- Database: PostgreSQL + PostGIS (nearby communities/services). Redis (cache/queues).
- Realtime: WebSockets/SSE for feed, votes, chat, map, notifications.
- Storage: S3/R2 (media) + IPFS (public evidence/receipts, content-addressed).
- Services split: identity, governance, treasury, marketplace, notification, audit, JARVIS tool-gateway.

2.3 Blockchain (Ethereum ecosystem)

- Solidity contracts: Society DAO (membership/roles), Voting (proposal, quorum, veto), Treasury (multi-sig, milestone disbursement), Project Registry (budgets, receipt hashes, verification).
- Start on Polygon Amoy / Base testnet (free); L2 mainnet for production. Account abstraction for smooth UX.
- Chain holds ONLY proofs: credential proofs, vote commitments, results, treasury events, receipt hashes, rule-version hashes. NEVER raw CNIC, phone, address, keys, biometrics, private messages.
- Index events (The Graph); anchor IPFS hashes on-chain for tamper-proof public records.

2.4 Decentralized data — the honest model

Phones do local-first storage + peer-to-peer sync (CRDTs); private keys and sensitive data stay encrypted on-device. BUT phones cannot be the only database — devices are lost, damaged, offline or reset. Shared votes, funds and public records need encrypted replication, consensus, backup and recovery. Model = hybrid: private on-device · public replicated + chain-anchored proofs.

2.5 JARVIS (intelligence layer)

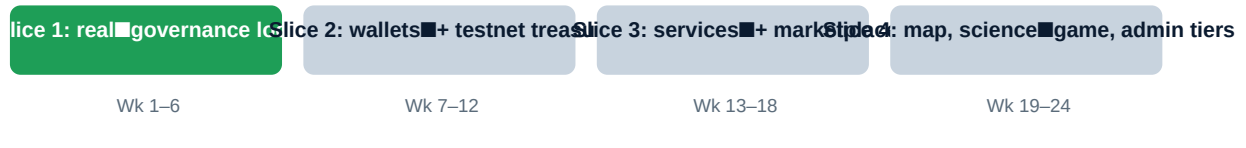
- Permission-scoped context service (communities, skills, open votes, wallet — user-approved only).
- Actions: explain/compare/draft proposals, summarize ledger, match services, translate, daily brief.
- Governance-safe: advises only — cannot vote or move funds. Requires confirmation before any side-effect.
- Add RAG over the founder's books/mission so it reasons in his worldview + integrity rules.

Core data model (starter)

Entity	Key fields
User	id, name, phone, email, verify_level, wallet_addr, skills, city, geo
Community	id, name, type, level, geo, constitution_ver, treasury_addr, admins
Membership	user_id, community_id, role, status, joined_at
Proposal	id, community_id, author, scope, status, budget, quorum, veto_pct, chain_ref
Vote	proposal_id, user_id, choice, weight, sig, committed_hash
Project	id, proposal_id, milestones[], receipts[], verified_by, status
LedgerEntry	community_id, actor, type, amount, ref, receipt_hash, ts
Service	provider_id, type, geo, price, status (requested->paid)

Build phases (module-by-module, not one AI dump)

ROADMAP



First vertical slice (prove it real):

Signup → Verify → Discover community → Join → Raise issue → JARVIS explanation → Vote → Fund → Track → Verify. Build this end-to-end on a real backend + testnet before widening. This is the demo that wins the team and the round.

Recommended order

- 1. Design system + app shell (RN/Expo + web). 2. Auth + onboarding (real OTP, location, purpose).
- 3. Communities + membership + treasury. 4. Proposals + discussion + voting (DB first).
- 5. Wire voting/treasury to a testnet contract. 6. Projects + receipts + verification.
- 7. JARVIS context service + actions. 8. World map + tiered scope. 9. Marketplace.
- 10. Emergency. 11. Science Lab / gaming. 12. Admin panels (society→global). 13. Audit + pilot.

Security, roles & the gaming layer

- Roles: member, community admin, city/country/global coordinator, system admin — every admin action audit-logged.
- Global admin cannot secretly control votes or funds — enforced by contracts + public logs.
- Threat model: Sybil resistance (verified identity + trust graph), replay/dispute processes, key recovery.
- Identity: W3C DID + Verifiable Credentials; hardware-backed keys (Keystore/Keychain); never publish raw ID.

Gaming — the Humanity Lab / Science Game

- Phase 1 (now): interactive challenge cards + simple simulations (water, traffic, energy, climate, orbital physics).
- Phase 2: multiplayer teams, AI mentor, scoring, scientist review, token/NDS rewards, implementation pathway.
- Phase 3: a Minecraft/GTA-scale world running real physical laws — a separate product that feeds real solutions back into communities. Turns 2B+ gamers into distributed problem-solvers.

Team, cost drivers & honest blockers

Minimum serious team

- Technical co-founder · React Native + web engineers · backend engineers · blockchain/Solidity engineer ·
- AI/JARVIS engineer · data engineer · UI/UX + design-system · DevOps/SRE · security engineer · QA ·
- identity/KYC specialist · financial-compliance counsel · governance designer · community ops · localization.

What money & external things are required (cannot be faked)

- Developer accounts (Google \$25, Apple \$99), Twilio/SMS, KYC provider, map provider, production DB/servers.
- Smart-contract + security audits before real funds. Payment/crypto-custody partners (licensed).
- Legal: privacy policy, terms, financial + identity compliance (jurisdiction-specific). App-store review.

Acceptance criteria for 'real'

- 50+ real pilot members complete the full loop with real (testnet) funds and public receipts.
- Every vote/treasury action verifiable on-chain; every project has receipt + verification.
- Independent security review passed on the first vertical slice before any mainnet/real money.